

DAY 5 ... TASK 5:

ABSTRACT THINKING & MAGICAL POWERS IN PYTHON

Hide the Complexity, Reveal the Essentials Abstraction, Dunder Magic & Python's Parent of All

✓ Assign Task 5

Theoretical Assignment

1. Abstraction

- Define abstraction in your own words.
- Why is abstraction important in object-oriented programming?
- How is abstraction different from encapsulation? Give an example.
- Explain the role of `@abstractmethod` and the ABC class from Python's `abc` module.
- Real-world example of abstraction (ATM machine, car interface, remote control, etc.).

2. Special (Dunder) Methods

- What are dunder methods? Why are they called "magic" methods?
- Explain the following special methods with examples:
 - `__init__`
 - `__str__` vs `__repr__`
 - `__eq__`, `__lt__`, `__add__`
 - `__call__`
 - `__del__`
- What is operator overloading, and why is it useful?

3. The object Class

- What is the object class in Python?
 - How do all classes inherit from it by default?
 - How can you use built-in methods from it like `object.__eq__()`?
-

Practical Coding Task

➤ Scenario:

Step-by-step instructions:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): pass

    @abstractmethod
    def perimeter(self): pass

    def __str__(self):
        return f"This is a {self.__class__.__name__}"
```

Now create **two subclasses**:

1. Circle(radius)
2. Rectangle(length, width)

Each class must:

- Implement area() and perimeter() methods
- Use __str__() to return a readable string
- Override __eq__() to compare two shapes based on their area
- Use __repr__() to show debug-style info
- Implement __call__() so the object acts like a function returning name and area

➔ Bonus Task:

- Create a list of mixed shape objects and loop through them, displaying area/perimeter using polymorphism.
- Use isinstance() and issubclass() to demonstrate abstraction in action.

➔ Bonus Tip

"Dunder methods make your classes act like built-in Python types. When used wisely, they make your code clean, powerful, and Pythonic."

 Deadline:

*Submit before: **Tomorrow night by 11:59 PM** (before Day 6 starts)*

Final:

Once you finish this task, send the message:

“ **Task 5 Completed**”

You're doing great — keep leveling up!